МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ федерального государственного
бюджетного образовательного учреждения высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ
Г.В.ПЛЕХАНОВА» Техникум Пермского института (филиала)

**Практическая работа №2**

по дисциплине: Поддержка и тестирование программных модулей

Руководитель:
Д.Б.Берестов /И.О.Фамилия/
“__30__”_октября_2026
Исполнитель:
А.В.Язев /И.О.Фамилия/
“__30__”октября__2026

Пермь,2026

# Оглавление

1. Цель и задачи работы ....3
2. Объект тестирования....3
3. Структура решения ....4
4. Ход выполнения работы ....5
4.1. Создание архитектуры решения ....5
4.2. Реализация бизнес-логики ....5
4.3. Разработка пользовательского интерфейса....6
4.4. Написание модульных тестов....7
4.5. Тестирование базовой функциональности....7
4.6. Исправление выявленных дефектов ....9
5. Результаты работы....11
6. Заключение ....11
Вывод:....14

# 1. Цель и задачи работы

В ходе выполнения данной практической работы были освоены принципы

разработки приложений с использованием технологии WPF (Windows

Presentation Foundation) и принципы модульного тестирования для управляемого

кода в среде .NET

Ключевыми задачами являлись: Получение навыков создания

многоуровневой архитектуры приложения с разделением бизнес-логики и пользовательского интерфейса; Разработка библиотеки классов для выполнения

арифметических операций; Создание графического пользовательского

интерфейса с использованием WPF и XAML; Написание модульных тестов с применением фреймворка MSTest; Выявление и исправление программных

дефектов посредством тестирования; Практическая отработка принципов TDD (разработка через тестирование)

# 2. Объект тестирования

Объектом исследования выступил класс Calculator, реализующий

функционал арифметического калькулятора. Класс включает следующие

методы:

1. Add(double a, double b) — сложение двух чисел
2. Subtract(double a, double b) — вычитание двух чисел
3. Multiply(double a, double b) — умножение двух чисел
4. Divide(double a, double b) — деление двух чисел
5. Power(double a, double exponent) — возведение в степень

Основное внимание было уделено корректности выполнения

арифметических операций и обработке граничных условий, таких как деление на ноль.

Интерфейс ICalculator обеспечивает возможность дальнейшего расширения функционала и упрощает тестирование за счет внедрения зависимостей.

## 3. Структура решения

Работа выполнялась в среде Visual Studio 2022. Решение Calcul состояло из трех проектов, целевой платформой для всех выбран .NET 8.0:

1. Calculator.Core — библиотека классов (.NET Class Library), содержащая тестируемую бизнес-логику: Интерфейс ICalculator.cs; Класс Calculator.cs с реализацией арифметических операций

2. Calculator.WPF — приложение WPF (Windows Presentation Foundation), реализующее пользовательский интерфейс:

- Файлы разметки XAML для описания интерфейса
- Code-behind файлы с обработчиками событий
- Монохромный дизайн в темных тонах

3.Calculator.Tests — проектмодульных тестов на базе фреймворка MSTest:

- Тестовые методы для проверки корректности каждой операции
- Проверка обработки исключительных ситуаций

Настройка зависимостей:

В проекте тестов была подключена зависимость MSTest.TestFramework версии 3.1.1 и настроена ссылка на проект Calculator.Core, что обеспечило доступ тестов к классам основной библиотеки.

Проект Calculator.WPF содержит ссылку на Calculator.Core для использования бизнес-логики в приложении.

# 4. Ход выполнения работы

## 4.1. Создание архитектуры решения

На первом этапе было создано пустое решение Calculator_Lazukov. В решение добавлены три проекта: Calculator.Core (Class Library, .NET 8.0), Calculator.WPF (WPF Application, .NET 8.0), Calculator.Test (MSTest Test Project,
.NET 8.0)

## 4.2. Реализация бизнес-логики

В проекте Calculator.Core создан интерфейс ICalculator:

```
public interface ICalculator
{
double Add(double a, double b);
double Subtract(double a, double b);
double Multiply(double a, double b);
double Divide(double a, double b);
double Power(double a, double exponent);
}
```

Реализация класса Calculator:

```
public class Calculator : ICalculator
{
public double Add(double a, double b) => a + b;
public double Subtract(double a, double b) => a - b;
public double Multiply(double a, double b) => a * b;
public double Divide(double a, double b)
```

```
{
if (b == 0)
throw new DivideByZeroException("Деление на ноль недопустимо");
return a / b;
}
public double Power(double a, double exponent) => Math.Pow(a, exponent);
}
```

## 4.3. Разработка пользовательского интерфейса

Интерфейс приложения разработан с использованием XAML (Extensible Application Markup Language). Применен монохромный дизайн в темных тонах.

Структура окна

Верхняя часть: текстовое поле TextBlock для отображения текущего значения

Нижняя часть: сетка кнопок 4×5 для ввода цифр и операций

Стили кнопок

Создано три стиля для различных типов кнопок:

1. NumberButton — кнопки цифр (фон #2D2D2D)
2. OperatorButton — кнопки операций (фон #4A4A4A)
3. EqualsButton — кнопка равно (фон #606060)

Каждый стиль включает триггеры для визуальной обратной связи:

IsMouseOver — изменение фона при наведении

IsPressed — изменение фона при нажатии

Обработка событий

Для каждой кнопки назначены обработчики событий Click:

Number_Click — ввод цифр

Decimal_Click — ввод десятичной точки

Add_Click, Subtract_Click, Multiply_Click, Divide_Click — выбор

операции

Equals_Click — выполнение вычисления

Clear_Click — очистка

Percent_Click — вычисление процента

PlusMinus_Click — смена знака.

## 4.4. Написание модульных тестов

В проекте Calculator.Tests создан класс тестов CalculatorTests с использованием атрибутов MSTest.

```
[TestClass]
public class CalculatorTests
{
private ICalculator _calc;
[TestInitialize]
public void Setup()
{
_calc = new Calculator.Core.Calculator();
}
}
```

Атрибут [TestInitialize] обеспечивает создание нового экземпляра калькулятора перед каждым тестом, что гарантирует независимость тестов друг от друга.

## 4.5. Тестирование базовой функциональности

Тест 1: Проверка операции сложения

```
[TestMethod]
public void Add_PositiveNumbers_ReturnsCorrectResult()
{
double result = _calc.Add(2, 3);
```

```
Assert.AreEqual(5.0, result, 0.0001);
}
```

Тест 2: Проверка операции вычитания

```
[TestMethod]
public void Subtract_PositiveNumbers_ReturnsCorrectResult()
{
double result = _calc.Subtract(5, 4);
Assert.AreEqual(1.0, result, 0.0001);
}
```

Тест 3: Проверка операции умножения

```
[TestMethod]
public void Multiply_PositiveNumbers_ReturnsCorrectResult()
{
double result = _calc.Multiply(3, 4);
Assert.AreEqual(12.0, result, 0.0001);
}
```

Тест 4: Проверка операции деления

```
[TestMethod]
public void Divide_PositiveNumbers_ReturnsCorrectResult()
{
double result = _calc.Divide(10, 4);
Assert.AreEqual(2.5, result, 0.0001);
}
```

Тест 5: Проверка возведения в степень

```
[TestMethod]
public void Power_PositiveExponent_ReturnsCorrectResult()
{
double result = _calc.Power(2, 3);
Assert.AreEqual(8.0, result, 0.0001);
```

```
}
```

Тестирование граничных условий

```
[TestMethod]
public void Divide_ByZero_ThrowsDivideByZeroException()
{
Assert.ThrowsException<DivideByZeroException>(() => _calc.Divide(5, 0));
}
[TestMethod]
public void Power_ZeroExponent_ReturnsOne()
{
double result = _calc.Power(5, 0);
Assert.AreEqual(1.0, result, 0.0001);
}
[TestMethod]
public void Add_NegativeNumbers_ReturnsCorrectResult()
{
double result = _calc.Add(-2, -3);
Assert.AreEqual(-5.0, result, 0.0001);
}
```

## 4.6. Исправление выявленных дефектов

Проблема с разделителем десятичных дробей

При вводе числа с точкой и выполнении операции калькулятор выдавал ошибку

Причина : В русской локали десятичный разделитель – запятая, а не точка. Метод double.TryParse не мог распарсить строку "5.5"

Решение : Реализован вспомогательный метод ParseNumber, который заменяет точку на запятую перед парсингом :

```
private double ParseNumber(string input)
```

```
{
string normalizedInput = input.Replace('.', ',');
double.TryParse(normalizedInput, NumberStyles.Number,
CultureInfo.CurrentCulture, out double result);
return result;
}
```

# 5. Результаты работы

По итогам выполнения работы создано полнофункциональное WPFприложение «Калькулятор» с модульной архитектурой и покрытием модульными тестами.

Реализованный функционал:

1. Арифметические операции: сложение, вычитание, умножение, деление
2. Дополнительные операции: возведение в степень, вычисление процента, смена знака
3. Работа с десятичными дробями
4. Обработка ошибочных ситуаций
5. Монохромный пользовательский интерфейс с визуальной обратной связью

Разработанные модульные тесты:

Создано 8 модульных тестов, покрывающих следующие сценарии:

1. Успешное выполнение операции сложения положительных чисел
2. Сложение отрицательных чисел
3. Вычитание положительных чисел
4. Умножение положительных чисел
5. Деление с получением дробного результата
6. Обработка деления на ноль (выброс исключения)
7. Возведение в положительную степень
8. Возведение в нулевую степень

# 6. Заключение

В ходе выполнения практической работы получены следующие навыки и знания:

Технические навыки:

Работа с Visual Studio:

о Создание и настройка решений с несколькими проектами

о Управление зависимостями между проектами

Использование обозревателя решений и навигация по коду

Разработка на C# и .NET:

Создание библиотек классов

Реализация интерфейсов и абстракций

Обработка исключительных ситуаций

Работа с пространствами имен

WPF-разработка:

Создание пользовательских интерфейсов с использованием XAML

Определение стилей и шаблонов элементов управления

Работа с триггерами для интерактивности

Обработка событий пользовательского интерфейса

Привязка данных и code-behind подход

Модульное тестирование:

Использование фреймворка MSTest

Применение атрибутов [TestClass] и [TestMethod]

Использование утверждений: Assert.AreEqual, Assert.ThrowsException, Assert.Fail

Инициализация тестов через [TestInitialize]

Написание тестов для граничных условий

Методологические аспекты:

Принципы чистой архитектуры:

Разделение бизнес-логики и слоя представления

Использование интерфейсов для слабой связанности

Возможность независимого тестирования компонентов

TDD:

Написание тестов до реализации функционала

Рефакторинг кода на основе результатов тестов

Создание защитного механизма от регрессий

Отладка и исправление дефектов:

Анализ сообщений об ошибках компиляции

Выявление логических ошибок через тестирование

Пошаговая отладка с использованием точек останова

Исправление проблем совместимости платформ

Практическая значимость:

Внедрение модульного тестирования позволило:

Автоматически обнаружить ошибку в обработке десятичных дробей

Обеспечить корректность арифметических операций

Создать документацию к коду через тестовые сценарии

Упростить дальнейшее расширение функционала

Возможности расширения проекта:

1. Добавление научного режима (тригонометрические функции, логарифмы)
2. Реализация истории вычислений
3. Добавление памяти калькулятора (M+, M-, MR, MC)
4. Поддержка работы с комплексными числами
5. Внедрение паттерна MVVM для улучшения архитектуры WPFприложения
6. Добавление тем оформления (светлая/темная тема)

# Вывод:

Выполненная работа продемонстрировала важность комплексного подхода к разработке программного обеспечения, включающего проектирование архитектуры, реализацию бизнес-логики, создание удобного пользовательского интерфейса и обязательное модульное тестирование.

Полученные навыки являются фундаментальными для профессиональнойразработки приложений на платформе .NET.